

# **Cloud native temperature excursion and compliance platform**

|                |                                                             |
|----------------|-------------------------------------------------------------|
| <b>SHAAMAK</b> | <b>Shipment, Simulation<br/>and Telemetry Service</b>       |
| <b>RIYA</b>    | <b>Excursion Service, Alert<br/>Service, Report Service</b> |
| <b>RUSSEL</b>  | <b>User Authentication</b>                                  |
| <b>PRANAV</b>  | <b>Data Flow Diagram ,<br/>Apache Kafka</b>                 |
| <b>RONAK</b>   | <b>Database</b>                                             |

# Shipment Microservice

## Working:

- Expose REST API for Logistics Managers to create shipments.
- Bind physical sensor\_id to logical shipment\_id.
- Store compliance rules (min\_temp\_limit, max\_temp\_limit).
- Expose endpoint for Warehouse Receivers to update shipment status.
- Broadcast event when status changes to 'In-Transit' to wake up the Simulator.

# Simulation Microservice

## Working:

- Create multiple asynchronous workers to handle each active sensor at the same time.
- Generate JSON telemetry payloads(Timestamp, Sensor ID, GPS, Temperature).
- Inject real-world network "jitter"
- Publish message directly to the Kafka topic.

# Telemetry Microservice

## Working:

- Subscribes to Kafka topic
- Pull messages off the queue at a steady, controlled rate.
- Buffer incoming messages in memory for a fraction of a second.
- Execute a batch INSERT transaction to save database CPU load.



# Excursion detection service

## Steps (Implementation)

- Consume data from Kafka (telemetry-events)ie it should subscribe to that topic
- Fetch temperature limits from DB
- Compare temperature with limits
- Store excursion in TimeScaleDB
- Publish event to excursion-events

## Communication

- Input → Kafka
- Output → Kafka
- DB → TimeScaleDB

# Excursion detection service

## Contract Testing

- Validate event schema before publishing
- Ensure fields: deviceId, temperature, timesta

**[Kafka Topic: telemetry-events]**

↓ (subscribe)

**[Excursion Service]**

↓ (process: check limits)

**[TimeScaleDB: store excursion]**

↓ (publish)

**[Kafka Topic: excursion-events]**

# Alert Management Service

## Steps (Implementation)

- Subscribe to Kafka topic excursion-events
- Receive excursion event message
- Analyze temperature deviation
- Decide severity (WARNING / CRITICAL)
- Store alert in TimeScaleDB
- Expose API /alerts for dashboard

## Communication

- Input → Kafka (excursion-events)
- Output → REST API (/alerts)
- DB → TimeScaleDB



# Alert Management Service

## Contract Testing

- Validate incoming event format (deviceId, temperature, status)
- Maintain fixed API response structure

**[Kafka: excursion-events]**

↓ (subscribe)

**[Alert Service]**

↓ (process logic)

**[TimeScaleDB: alerts table]**

↓

**[REST API: /alerts]**

↓

**[Dashboard]**

# Reporting Service

## Steps (Implementation)

- Fetch data from TimeScaleDB (excursions + alerts)
- Process data (average temp, total excursions, alerts)
- Generate report object
- Expose API /reports

## Communication

- Input → TimeScaleDB
- Output → REST API (/reports)

# Reporting Service

## Contract Testing

- Maintain fixed report format (avgTemp, totalExcursions)
- Ensure consistent API response

**[TimeScaleDB]**

↓ (query)

**[Reporting Service]**

↓ (process data)

**[REST API: /reports]**

↓

**[Dashboard]**

# Dashboard Service

## Steps (Implementation)

- Call Alert API (/alerts)
- Call Reporting API (/reports)
- Display alerts and analytics
- Refresh data periodically

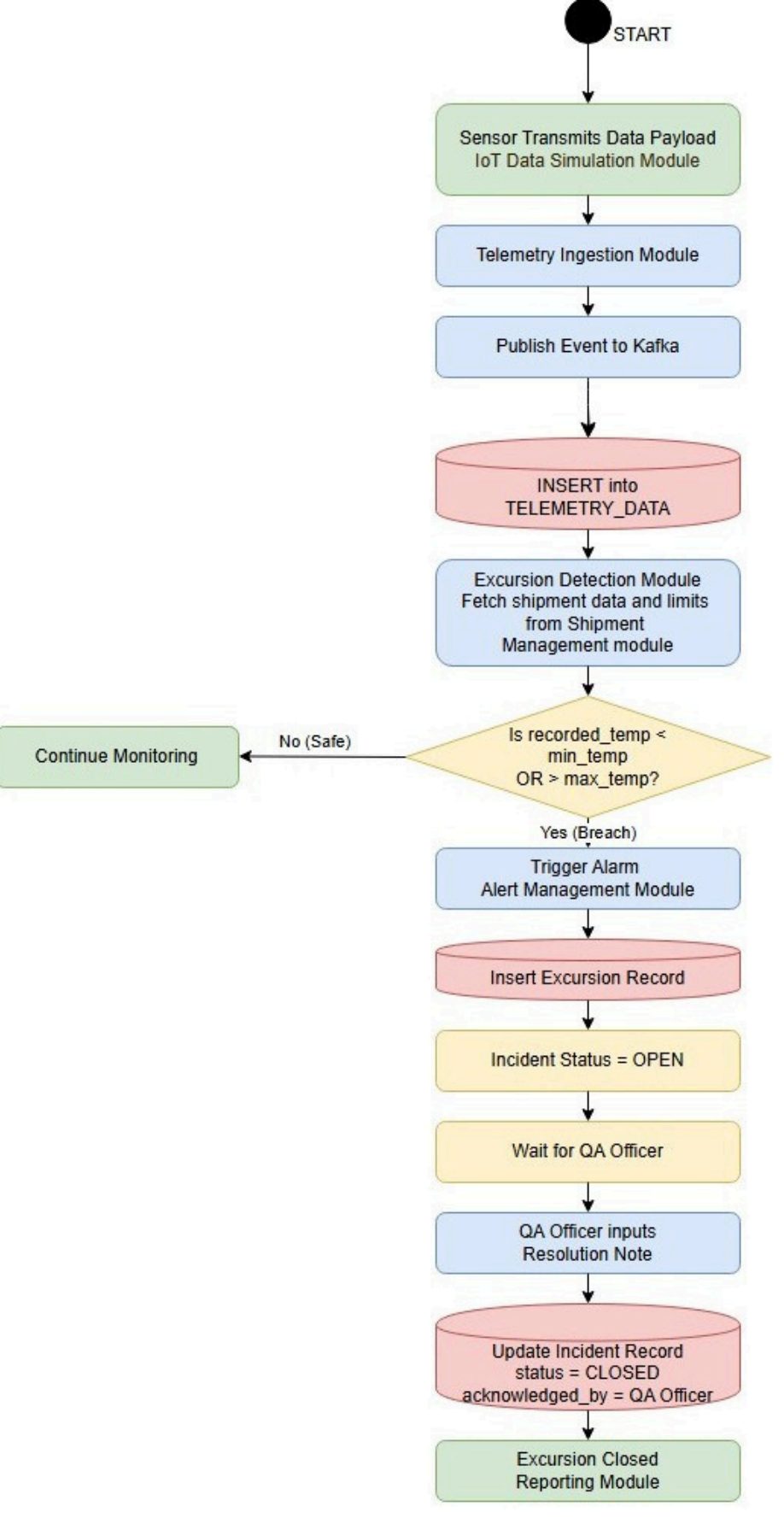
## Communication

- Input → REST APIs
- No DB / Kafka

## Contract Testing

- Validate API responses
- Ensure required fields exist

# Activity diagram :





# User Authentication:

## System Users:

### 1. Client :

- Access Dashboard Service
- View shipment data & status

### 2. QA Officer

- Access Alert Management Service
- Monitor and resolve temperature excursions

### 3. Compliance Officer

- Access Reporting Service
- View reports and ensure regulatory compliance

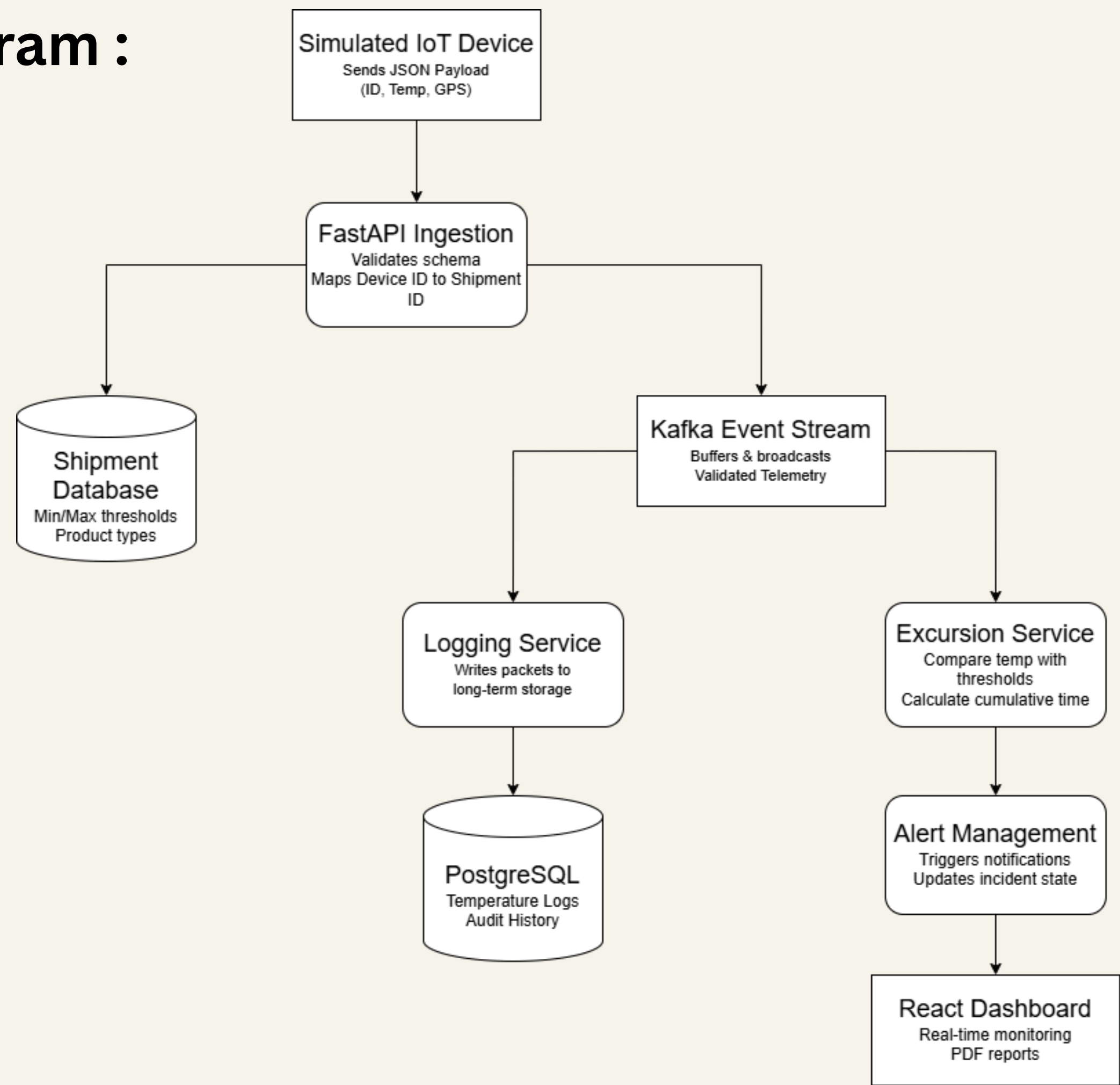
## Implementation:

- User logs in with email & password
- Server verifies credentials
- JWT token is generated
- Token is sent to user
- Token is used for accessing protected APIs

## Security Features:

- password hashing
- JWT tokens with expiry time
- Protected API routes
- Role-based authorization for modules

# Data Flow Diagram :





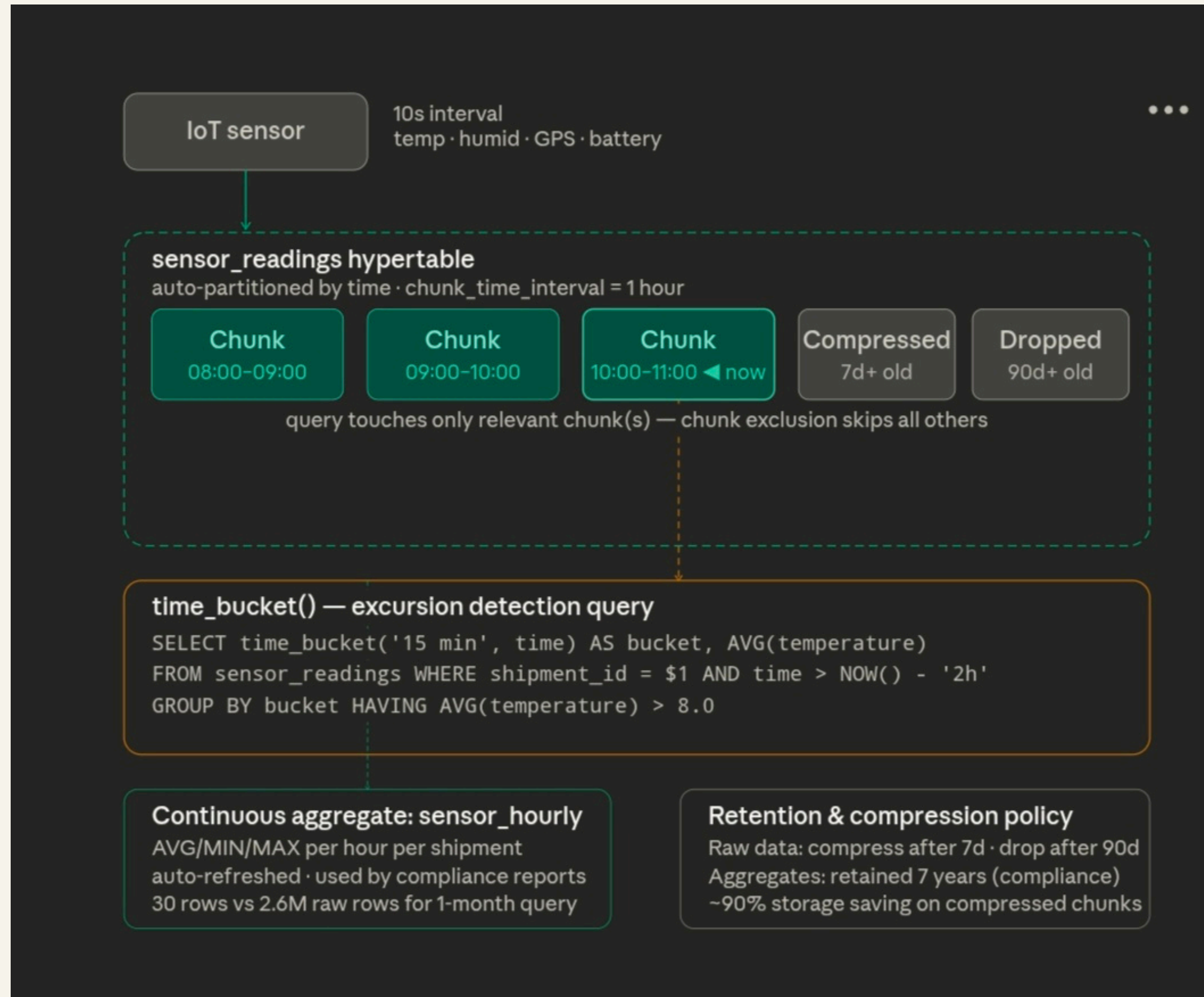
# Apache Kafka

- Introduction
- Problem of Tight Coupling
- The Producer Consumer logic
- Reliability through Persistence
- Fault tolerance and logic of offsets
- Scalability and Design for the future

# Implementation

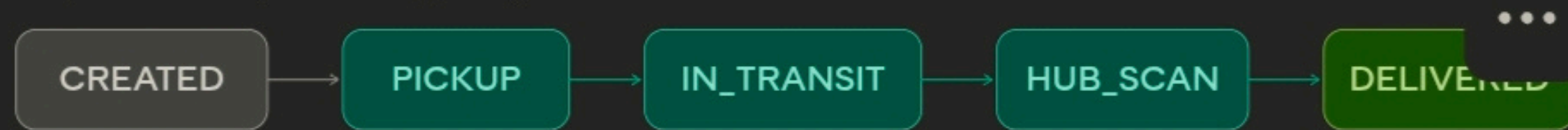
- The API Layer
- Setting up the Broker & Topics
- Developing the Logic
- The Frontend

# TimescaleDB — the sensor memory

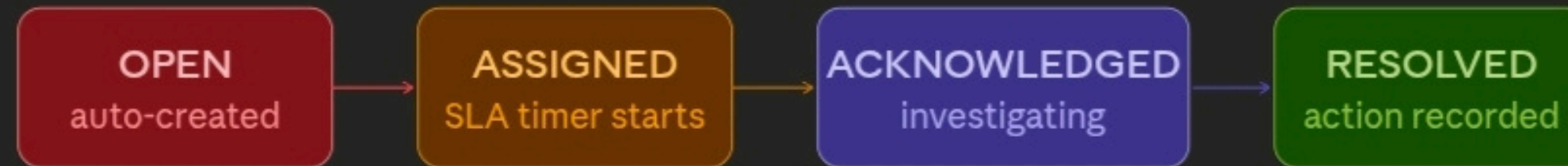


# PostgreSQL

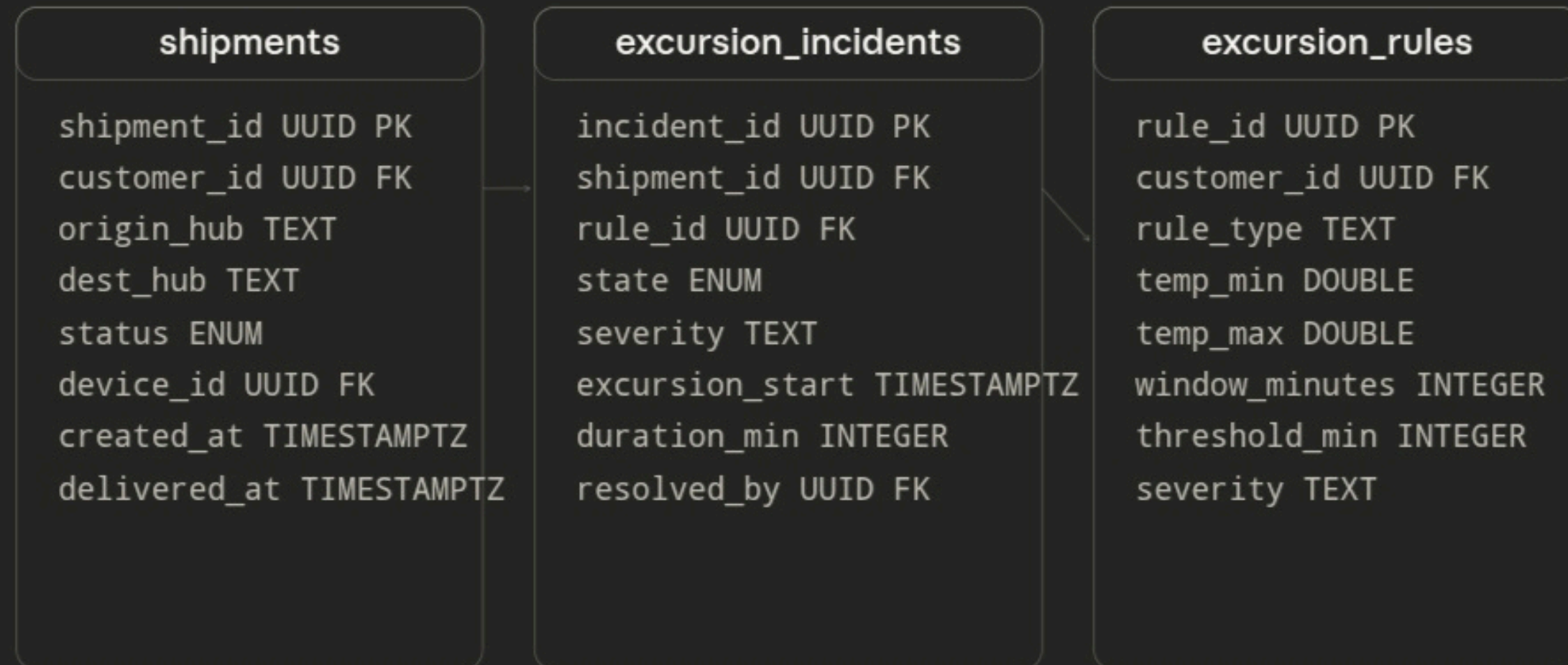
## Shipment lifecycle (PostgreSQL state machine)



## Incident workflow — triggered on excursion

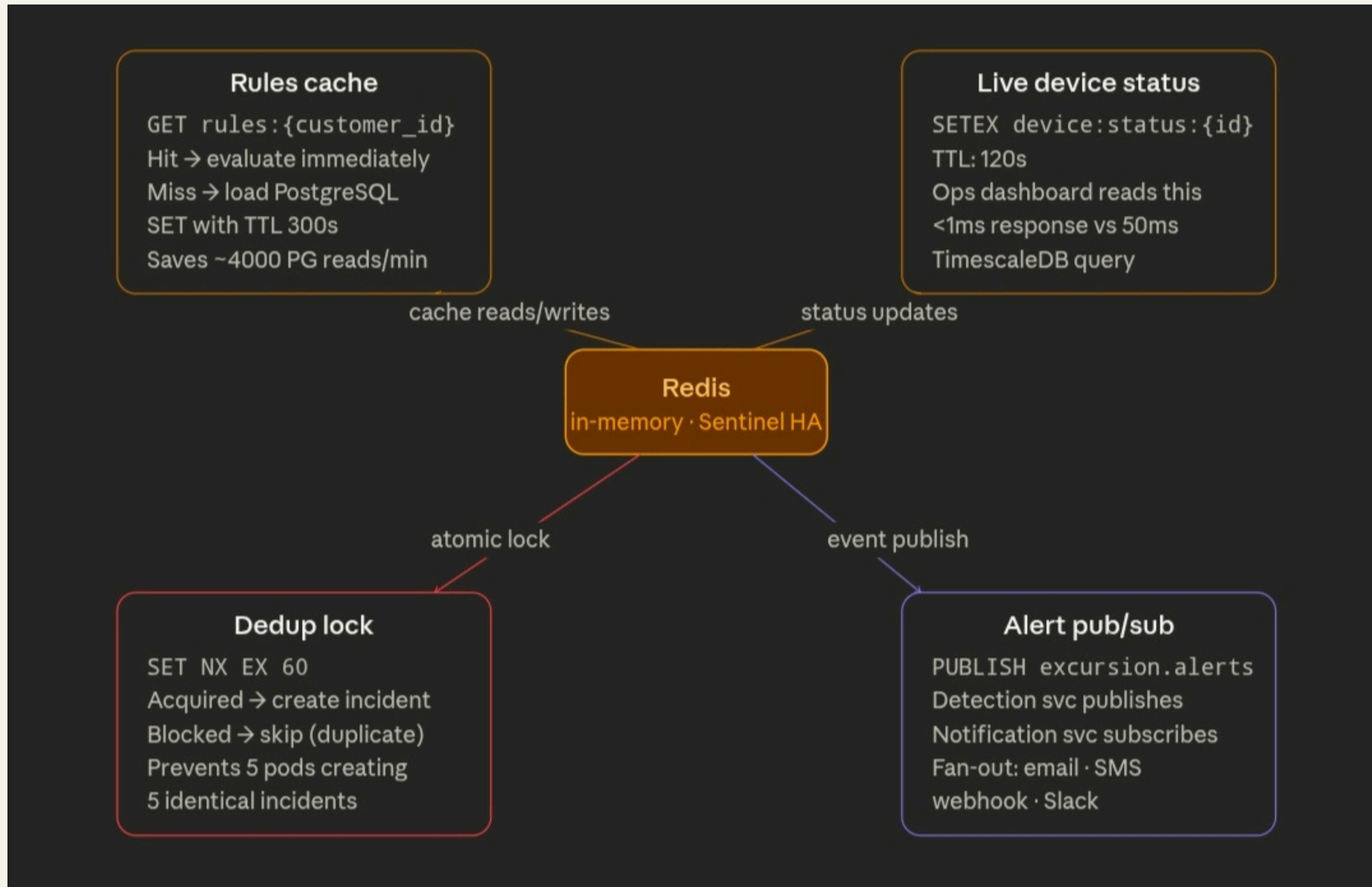


## Key tables



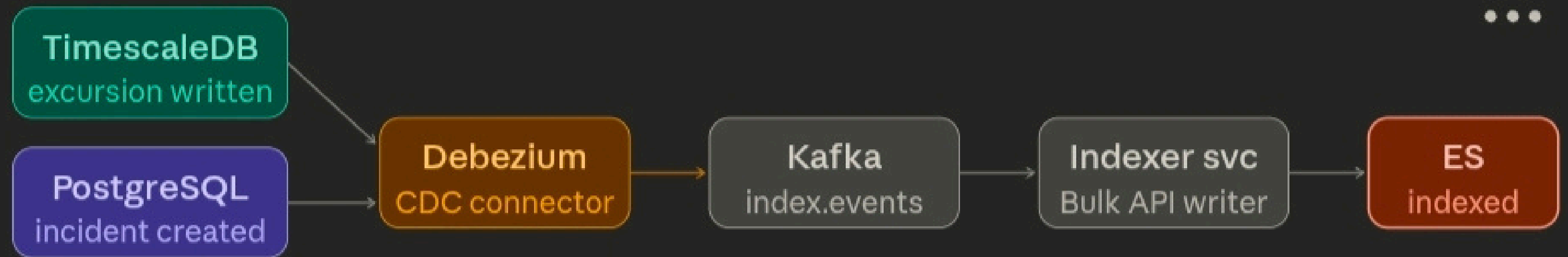


# Redis — the fast coordinator

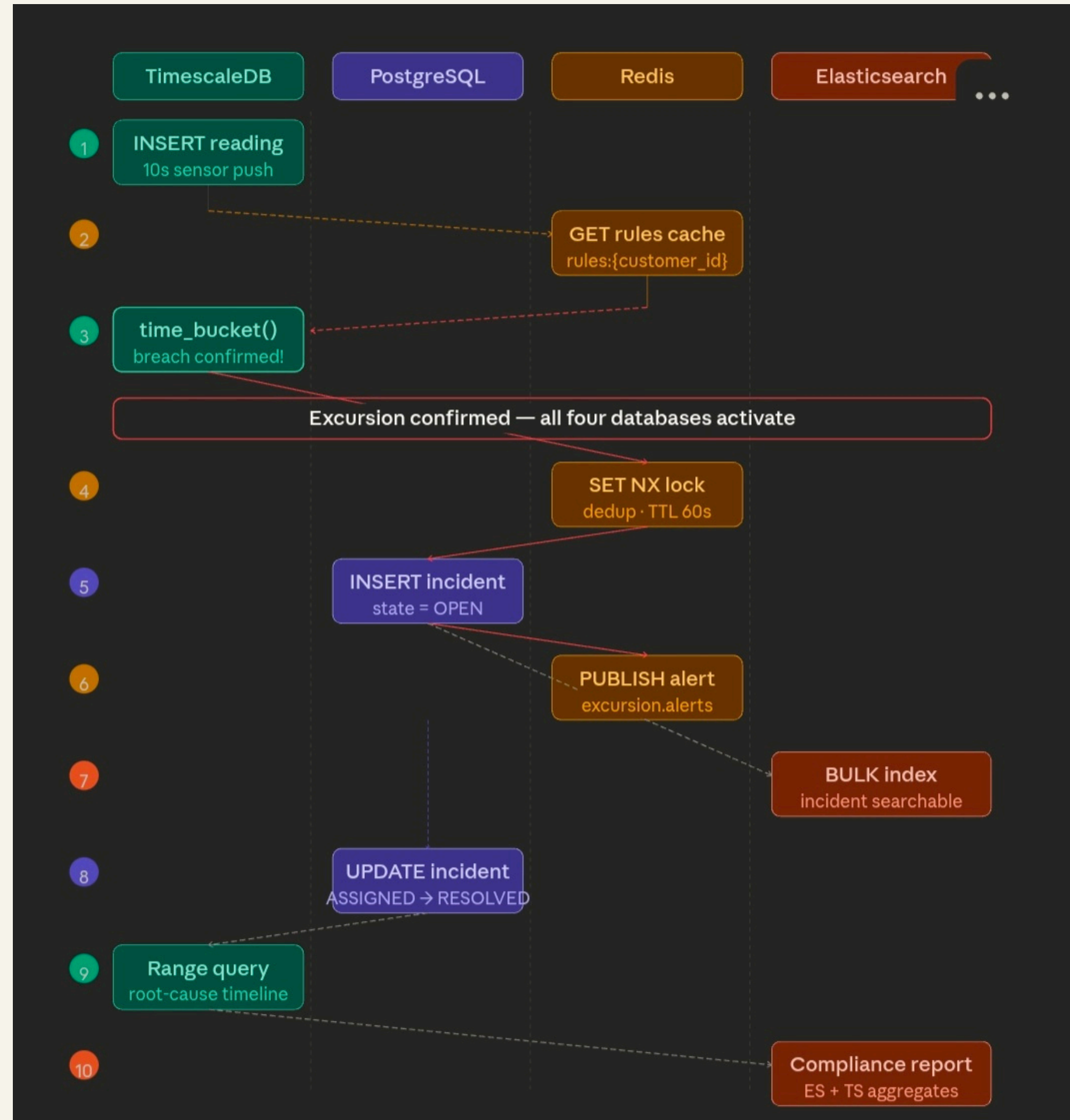


# Elasticsearch — the investigator's window

Indexing pipeline — CDC via Debezium



The combined picture — all four working together during a live excursion



## Work to be done next week

- Simulation Script
- UI Mockup
- Implementing Kafka
- Check contract testing